Software Requirements Specification (SRS)
Low Latency Trading Exchange

Manjot Singh (24116048)
Anjan Singla (24114012)
Adityavardhan Singh (24322005)
Mayank Rana (24115101)
Abhinav Shresth (24115008)

September 30, 2026

Contents

1 Introduction **2**
1.1 Definitions . . . 2

2 Scope **2**
2.1 Included Features . . . 2
2.2 Out-of-Scope Features . . . 3

3 System Architecture Overview **4**
3.1 Single-Threaded Dispatcher Loop: . . . 4
3.2 Decoupled Worker Processing . . . 4

4 Communication Protocol Specification **5**
4.1 TCP Transport Framing and Header Conventions . . . 5
4.2 Client Request Specifications (TCP Channel) . . . 5
4.3 Server Response Specifications (TCP Channel) . . . 6
4.4 UDP Market Data Feed Specification . . . 6

5 Specific Requirements **7**
5.1 Functional Requirements . . . 7
5.2 Non-Functional Requirements . . . 9

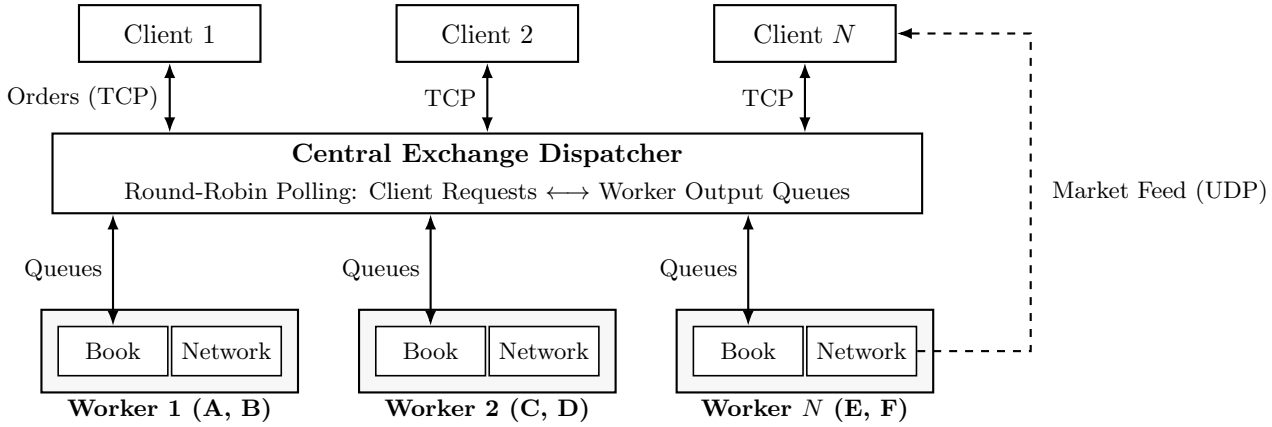


Figure 1: System Architecture: Client Request Dispatch and Direct UDP Feed Model.

1 Introduction

This Software Requirements Specification (SRS) document details the functional and non-functional requirements for a hybrid low-latency stock exchange and order book simulation system. It specifies the protocol format, threading behavior, network transport interactions, and operational constraints for both the core matching engine and the participating client applications. This specification serves as the design baseline for system implementation, concurrency validation, and protocol testing.

1.1 Definitions

Ticker Group: A set of related tickers in an industry sector (e.g., Tech, Banking, Consulting).

Limit Order: An order to buy or sell a volume of an asset at a specified price or better.

Passive Order: An unexecuted limit order added to the order book.

Aggressive Order: An incoming order that matches immediately with resting liquidity.

Pro-Rata Matching: An allocation algorithm where incoming aggressive volume is divided among existing resting orders in proportion to each order's size relative to total resting volume at that price level.

L2 Market Feed: Information about aggregated volume across 3 price levels along with best Bid and Offer (BBO) data.

Dispatcher: Primary server component which accepts client connections and assigns orders to individual worker threads.

2 Scope

2.1 Included Features

The system contains the following core functional components:

TCP Client Protocol

An application layer protocol modeled on HTTP request-response pattern with lightweight JSON or delimited payloads operates on a reliable TCP stream. It supports:

- Submitting buy limit orders (`BID`) and sell limit orders (`ASK`) specifying ticker, price, and volume.
- Order cancellation (`CANCEL`) referencing a server-assigned identifier.
- Feed discovery via requests to list all supported ticker groups and the tickers assigned to each group.
- Subscription and unsubscription requests for UDP market feeds by ticker group.

TCP Server Responses

Responses returned over TCP contain a permanent server-generated `orderID`, the order state (`PARTIALLY_FILLED`, `COMPLETELY_FILLED`, `RESTING`, or `CANCELLED`), and an array of executed trade records if immediate matching occurred.

UDP based Market Feed

Direct transmission of top-3 bid/ask aggregated price and volume updates to client UDP sockets, bypassing the central dispatcher thread.

Demonstration Interface (TUI/GUI)

A client interface that connects to the TCP endpoint for manual or algorithmic order submission and displays real-time price time-series generated from the UDP multicast/broadcast stream.

2.2 Out-of-Scope Features

The following features are excluded from the current baseline and designated as future extensions:

- **Advanced Order Types:** Execution is restricted strictly to standard limit orders. Market orders, stop-loss orders, fill-or-kill (FOK), and immediate-or-cancel (IOC) orders are omitted.
- **Time-In-Force and Automatic Expiration:** Orders do not support duration attributes (such as Good 'Til Cancelled or Day orders)
- **Financial Derivatives:** No support for options, futures contracts, margin accounting, or derivative valuation models.
- **Full Book Depth Transmission:** The broadcast layer exposes only 3-level aggregated market depth (L2) and ignores deeper book levels or raw order-by-order (L3) queues.
- **Transport Layer Encryption:** Communication uses plaintext sockets without TLS/SSL to maintain minimal transport overhead. (HTTPS/TLS encapsulation is deferred to future work).
- **Dispatcher Thread Pooling:** The dispatcher runs as a single thread. Under high traffic, the dispatcher to a multi-threaded pool is treated as an architectural scaling path for future iterations which is out of scope for a network oriented project.
- **Persistent Storage and Crash Recovery:** All state resides in-memory. Database backing, write-ahead logging (WAL), and state reconstruction across crashes are excluded.

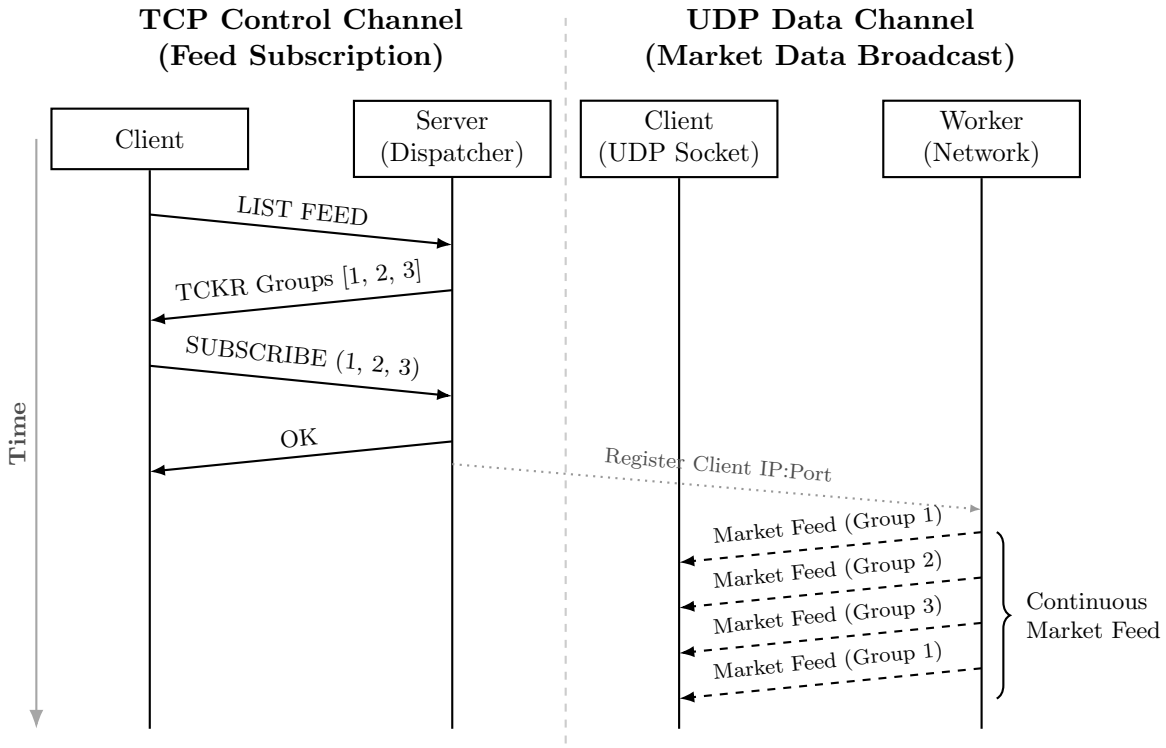


Figure 2: Dual-Channel Communication: TCP Feed Subscription and Direct UDP Market Feed.

3 System Architecture Overview

The system segregates low-latency matching from market data dissemination through asymmetric communication channels:

1. **Ingress and Acknowledgment Path (TCP):** Clients connect to a centralized Dispatcher. The Dispatcher accepts inbound connections, demultiplexes request frames, and routes order actions into the appropriate worker's task queue based on the requested ticker. Once a worker completes an execution cycle, the Dispatcher forwards the transaction status and trade list back to the client.

2. **Market Data Broadcast Path (UDP):** Workers maintain active client address tables for their assigned ticker groups. The worker's internal communication thread pushes L2 snapshots directly through a dedicated UDP socket to subscribers, eliminating dispatcher context-switching and socket serialization bottlenecks.

3.1 Single-Threaded Dispatcher Loop:

The non-blocking dispatcher runs an event loop that uses the Round-Robin Algorithm between reading client requests, routing tasks to worker input queues, reading completed worker outputs, and delivering formatted responses back to clients. This loop ensures that neither workers nor client requests are starved and consistent latency is maintained.

3.2 Decoupled Worker Processing

Each worker instance manages an assigned subset of order books and decouples order execution from market data dissemination:

– **Order Matching:** Ingestion, order book evaluation, and trade resolution operate continuously without stalling for network transmissions.

– **Direct Market Broadcast:** Top-3 aggregated depth generation and UDP broadcasting are handled independently, transmitting market updates directly from the worker to subscribed clients.

Market data updates shall be scheduled dynamically to ensure regular feed generation without degrading matching throughput. The broadcast trigger interval, T_{timeout}, is calculated using the running average duration of recent UDP transmissions plus an engineered safety threshold:

$$T_{\text{timeout}} = \overline{T}_{\text{UDP}} + \Delta t_{\text{safety}}$$

Here, $\overline{T}_{\text{UDP}}$ is the running average (EWMA) duration of outbound UDP transmissions and Δt_{safety} is a configurable margin to accommodate network jitter. When T_{timeout} elapses, the worker extracts an instantaneous Top-3 depth snapshot and dispatches the UDP feed concurrently with subsequent matching operations.

4 Communication Protocol Specification

4.1 TCP Transport Framing and Header Conventions

Requests and responses follow an HTTP/1.1 message structure. Each message consists of a start-line, standard headers and an optional JSON body:

1. Content-Length: (Mandatory for bodies) Specifies payload size in bytes to guarantee deterministic frame parsing over the continuous TCP stream.
2. Content-Type: Fixed as `application/json`.
3. Order-ID: Carried in the header for fast extraction on cancellation requests.

4.2 Client Request Specifications (TCP Channel)

Request	**HTTP Equivalent**	**Required Headers**	**Body Schema (JSON)**
BID	POST /order/bid	Content-Length Content-Type	{"ticker": "AAPL", "type": "LIMIT", "max_price": 180, "volume": 50}
ASK	POST /order/ask	Content-Length Content-Type	{"ticker": "AAPL", "type": "LIMIT", "min_price": 182, "volume": 30}
CANCEL	DELETE /order/cancel	Order-ID: <id>	*Empty*
LIST FEED	GET /feed/groups	*None*	*Empty*
SUBSCRIBE	POST /feed/subscribe	Content-Length Content-Type	{"groups": [1, 2, 3]}
UNSUBSCRIBE	POST /feed/unsubscribe	Content-Length Content-Type	{"groups": [2, 3, 4]}

Table 1: Client Request Message Formats over TCP.

Action	Status Code	Result State	Response Payload Schema (JSON)
Order Placement	200 OK	RESTING PARTIAL_FILL FILLED	{"order_id": 1042, "status": "PARTIAL_FILL", "remaining_vol": 20, "trades": [{"trade_id": 501, "price": 180, "vol": 30, "counterpart_id": 980}]}
Order Placement	200 OK	RESTING	{"order_id": 1043, "status": "RESTING", "remaining_vol": 50, "trades": []}
Order Cancel	200 OK	CANCELLED	{"order_id": 1042, "status": "CANCELLED"}
List Feed	200 OK	SUCCESS	{"groups": [{"group_id": 1, "name": "Tech", "tickers": ["AAPL", "GOOG"]}]}
Subscribe	200 OK	SUCCESS	{"subscribed": [1, 2, 3]}
Unsubscribe	200 OK	SUCCESS	{"unsubscribed": [2, 3]} *(Empty array if no matching active groups)*
Errors	400 / 404 / 422	ERROR	{"error": "ORDER_NOT_FOUND", "message": "ID 1042 does not exist or already filled"}

Table 2: Server Response Formats over TCP.

4.3 Server Response Specifications (TCP Channel)

4.4 UDP Market Data Feed Specification

Market data broadcasts are unacknowledged datagrams pushed directly from the worker to client endpoints. Each packet contains a sequence counter for client-side drop detection, a microsecond timestamp, and the Top-3 aggregate depth levels for a specific ticker:

Field	Data Type	Description
seq_num	Integer (uint64)	Monotonically increasing sequence number per group for loss detection.
timestamp	Integer (uint64)	Unix epoch timestamp in microseconds.
group_id	Integer (uint32)	Target ticker group identifier.
ticker	String (char[8])	Asset ticker symbol (e.g., "AAPL").
bids	Array of Pairs	Top 3 bid levels: [[price_1, vol_1], [price_2, vol_2], [price_3, vol_3]].
asks	Array of Pairs	Top 3 ask levels: [[price_1, vol_1], [price_2, vol_2], [price_3, vol_3]].

Table 3: UDP Market Feed Datagram Structure.

5 Specific Requirements

5.1 Functional Requirements

1. **FR-1: Connection Management**
   - The exchange dispatcher shall accept multiple concurrent client TCP connections on a designated port.
   - The protocol parser shall frame messages based on custom request structure desgin inspired from HTTP/1.1, utilizing the `Content-Length` header to read exact JSON payload boundaries from the TCP byte stream.
2. **FR-2: Order Intake and Assignment**
   - The system shall accept limit buy orders via `BID` and limit sell orders via `ASK`, with payloads specifying the target ticker, limit price, and volume.
   - The dispatcher shall validate input fields (ensuring positive non-zero price and volume) and submit the request to the corresponding worker instance handling that ticke.
   - The system shall generate and assign a strictly monotonic 64-bit integer (`order_id`) to uniquely identify the order across its lifecycle.
3. **FR-3: Pro-Rata Matching Algorithm**
   - Each worker's computation thread shall maintain independent bid and ask books for its assigned tickers, sorted in price-priority order.
   - When an incoming order crosses the spread, the matching engine shall execute volume against resting counterpart orders at that price level using pro-rata allocation:

$$\text{Vol}_i = \min\left(\text{Vol}_{\text{aggressive}}, \left\lfloor \text{Order Vol}_i \times \frac{\text{Vol}_{\text{aggressive}}}{\sum \text{Order Vol}_i} \right\rfloor\right)$$

   - Any fractional remainder from integer rounding shall be resolved using time priority among the resting orders at that price tier.
   - Unfilled volume of the incoming limit order shall be enqueued as a passive order in the book.
4. **FR-4: Order Cancellation**
   - The system shall accept cancellation requests via `CANCEL` bearing an `Order-ID` header.
   - If the referenced order is currently resting in the book, the worker shall remove it from the orderbook and return a `CANCELLED` state.
   - If the order does not exist or has already executed completely, the system shall return an `ERROR: 404 Not Found` error.
5. **FR-5: Execution Reporting and Response Delivery**
   - The dispatcher shall return a synchronous JSON response over the client's TCP socket upon completion of worker processing.
   - The payload shall report the generated `order_id`, the order state (`RESTING`, `PARTIAL_FILL`, or `FILLED`), remaining volume, and an array of executed trades containing matched volume, execution price, and assigned trade IDs.
6. **FR-6: Feed Discovery**

- The dispatcher shall process `LIST FEED` requests and return a complete list of ticker groups, associated group identifiers, and the specific tickers assigned to each group.

7. **FR-7: Feed Subscription and Unsubscription**
   - Clients shall submit `SUBSCRIBE` containing an array of group IDs over TCP. The dispatcher shall record the client's UDP endpoint and notify the corresponding workers.
   - Clients shall submit `UNSUBSCRIBE` containing an array of group IDs to withdraw subscriptions.
   - The unsubscription handler shall operate idempotently: it shall discard group IDs to which the client is not currently subscribed and return an array containing only the IDs that were actively removed.

8. **FR-8: Decoupled Dispatcher Event Loop**
   - The dispatcher shall execute a single-threaded, non-blocking polling loop that round-robins between servicing incoming client socket reads and forwarding resolved worker execution results back to clients.
   - Order routing shall be performed via lock-free or concurrent bounded worker queues to prevent thread stalling.

9. **FR-9: Adaptive Worker Yielding and Snapshot Extraction**
   - The worker computation thread shall track the dynamic timeout threshold:

     $$T_{\text{timeout}} = \overline{T}_{\text{UDP}} + \Delta t_{\text{safety}}$$

     where $\overline{T}_{\text{UDP}}$ is the running exponentially weighted moving average of recent UDP transmission durations, and Δt_{safety} is an engineered safety margin.
   - Following each processed order, if the elapsed time exceeds T_{timeout}, the computation thread shall temporarily release access to the order book.
   - The worker network thread shall capture a Top-3 depth snapshot (top 3 bids and top 3 asks with aggregated volume) and immediately release the book back to the computation thread.

10. **FR-10: UDP Market Data Feed**
    - The worker network thread shall serialize the Top-3 depth snapshot into a datagram containing a 64-bit monotonically increasing sequence number, a microsecond timestamp, the group ID, ticker symbol, and aggregated level data.
    - The worker shall transmit this datagram directly over UDP to the registered IP addresses and port numbers of subscribed clients, bypassing the central dispatcher thread.

11. **FR-11: Programmatic Client API Library**
    - The system shall expose a client-side shared library providing programmatic functions for session establishment, limit order placement, order cancellation, and group subscriptions.
    - The library shall run a background listener socket for UDP broadcasts, verify sequence continuity, and deliver decoded L2 market snapshots to algorithmic trading modules or higher-level applications via an event callback interface.

5.2 Non-Functional Requirements

1. **NFR-1: Processing Latency**
   - **Ingress Dispatch Latency:** The duration from TCP socket parsing by the dispatcher to insertion into a worker queue shall not exceed 50 microseconds under nominal load.
   - **Snapshot Extraction Bound:** The duration for which the worker network thread retains access to the shared book state to extract the Top-3 depth snapshot shall be bounded to under 5 microseconds, preventing latency spikes in order execution.
2. **NFR-2: Concurrency and Thread Isolation**
   - Worker instances shall operate without inter-worker locks; operations on separate ticker groups shall proceed in parallel without shared resource contention.
   - Outbound UDP network delays or socket write buffer stalls shall not block or delay the matching engine computation thread.
3. **NFR-3: Throughput Capacity**
   - The matching engine shall sustain a minimum processing rate of 250 order insertions, cancellations, or matches per second per worker instance on baseline hardware.
4. **NFR-4: Protocol Reliability and Fault Detection**
   - **TCP Connection Resilience:** Sockets experiencing EOF, connection reset, or malformed framing shall be terminated and cleaned up immediately by the dispatcher without affecting active sessions or book states.
   - **UDP Loss Transparency:** Because UDP datagrams are connectionless and unacknowledged, clients shall use the per-group 64-bit packet sequence number (`seq_num`) to detect network drops or out-of-order packets locally.
5. **NFR-5: Data Precision and Determinism**
   - All price points and volume counters shall be maintained as 64-bit unsigned integers to avoid floating-point rounding errors across different CPU architectures.
   - Matching outcomes, trade prices, and pro-rata share distributions shall be completely deterministic given the same sequence of inbound order timestamps.
6. **NFR-6: Resource Footprint and Persistence Constraints**
   - All order book allocations, queue buffers, and client tables shall reside strictly in volatile system memory (RAM).
   - The fast path shall execute without disk I/O, database transactions, or external logging systems to maintain low execution jitter.